Repository Workflow and Git-Flow Rules

1. GitHub organization and repositories

- **Organization:** the team uses the GitHub organization `seng321-team16`, owned by Ali Kaan Demiralp (GitHub: `dalikaan`). The other four members are organization members with the Write role through the team `developers`.
- **Evaluator access:** the course instructor (`nedima68`) and the two teaching assistants (`mhmttemin`, `goksuyldrm`) are added under Settings → Collaborators and teams with at least the Read role, so that repository content, commit history, issues and pull requests can be inspected. The repository link is stated in the submission description.
- **Repositories:** one monorepo `phishguard-trainer` with `/api`, `/web`, `/docs`, `/deploy`. A monorepo keeps one pipeline, one history and one place for the instructor to look.
- **Visibility:** the repository is public. Public repositories support branch protection and required reviews on the free GitHub plan, and the project is released under an open-source license. Secrets are never committed (see section 6).
- **Repo contents at creation:** `README.md`, `LICENSE` (Apache-2.0), `NOTICE`, `.gitignore`, `CODEOWNERS`, `CONTRIBUTING.md`, `CONTRIBUTIONS.md` (individual contributions), `SECURITY.md`, `docs/` (all Sprint 1 files), `api/openapi.yaml`.

2. Branching model (Git-Flow)

Branch	Purpose	Created from	Merges into
`main`	Always releasable, tagged releases only	n/a	n/a
`develop`	Integration branch, always green	`main`	n/a
`feature/<issue#>-<short-name>`	One story or task	`develop`	`develop` (PR)
`release/<x.y.z>`	Stabilise a sprint release	`develop`	`main` and back to `develop`
`hotfix/<issue#>-<name>`	Urgent fix on released code	`main`	`main` and `develop`

Example: `feature/12-login-endpoint`. Never commit directly to `main` or `develop`.

Release tags: `v0.1.0` (Sprint 1 docs), `v0.2.0` (Sprint 2), `v0.3.0` (MVP), `v1.0.0` (final). Each sprint submission references a tag.

3. Branch protection (configure in Settings → Branches/Rulesets)

Applies to **`main`** and **`develop`**:

- Require a pull request before merging.
- **Require 1 approving review** (minimum) from a team member other than the author. The Lead Architect is the default reviewer through CODEOWNERS, but review by Code Owners is not enforced, so that any member can review and the Lead Architect's own pull requests can be approved by another member.
- **Dismiss stale approvals** when new commits are pushed.
- **Authors cannot approve their own PR** (GitHub default).
- Require status checks to pass before merging: `build`, `lint`, `unit-tests` (and `openapi-lint` once the pipeline exists in Sprint 3). Require the branch to be up to date.
- Require conversation resolution before merge.
- Block force pushes and branch deletion.
- Require linear history (squash merge) on `develop` and `main`.
- Do not allow bypassing for admins, except a logged emergency (state the reason in the PR).
- Enable GitHub **secret scanning + push protection** and **Dependabot alerts**.

Review turnaround: a review request is answered within 48 hours. The Scrum Master follows up on open requests and records any delay as an impediment. The review rule is also the evidence trail of individual contributions.

4. Pull request rules

- One PR = one issue. Keep PRs small (target under 400 changed lines).
- PR title format: Conventional Commits, e.g. `feat(auth): add login endpoint (#12)`.
- PR description uses the template: *What / Why / How tested / Linked issue (`Closes #12`) / Screenshots (UI)*.
- Author checklist: tests added or updated, OpenAPI updated if the API changed, no secrets, docs updated.
- Reviewer checks: acceptance criteria met, tests exist, security basics (input validation, authorisation checks), readable code.
- Merge method: **squash merge** into `develop`; merge commit for `release/*` into `main`.
- Delete the feature branch after merge.

5. Issue conventions

- Every story/task is a GitHub Issue on the project board (**GitHub Projects**: columns Backlog → Ready → In Progress → In Review → Done).
- Title: `[US-07] Learner completes a training session` for stories, `[TASK] ...` for tasks, `[BUG] ...` for defects.
- Required fields: description, acceptance criteria (Given/When/Then), story points, priority, sprint milestone, assignee, epic label.
- Labels: `epic:auth`, `epic:training`, `epic:admin`, `epic:platform`, `type:story`, `type:bug`, `type:task`, `priority:must/should/could`, `security`, `blocked`.
- Milestones: `Sprint 1` … `Sprint 3`, `Final`.
- Definition of Ready: acceptance criteria, estimate, owner. Definition of Done: merged via reviewed PR, tests pass, OpenAPI/docs updated, acceptance criteria verified by QA.

6. Commit and security hygiene

- Conventional Commits (`feat`, `fix`, `docs`, `test`, `chore`, `refactor`). Sign commits if possible, but not mandatory.
- Real names or GitHub handles in commits, so contribution is attributable. No shared accounts.
- `.env` is git-ignored; ship `.env.example` only. If a secret is ever committed: rotate it first, then clean history.
- Enable 2FA on every GitHub account in the org (require it in org settings).

7. Repository bootstrap sequence

1. Create the organization and the `phishguard-trainer` repository with `LICENSE`, `NOTICE` and `README.md`.
2. Create the `main` and `develop` branches; `develop` is the default branch.
3. Configure the branch protection rules of section 3.
4. Add issue templates (story, task, bug), the pull request template and `CODEOWNERS`.
5. Create the GitHub Projects board and labels; create issues from `06-Initial-Product-Backlog.md`.
6. Invite all team members, require two-factor authentication, and add the instructor and teaching assistants as Read collaborators.
7. Tag the Sprint 1 submission as `v0.1.0`.

8. Scrum Ceremonies and Team Coordination

-The Scrum Master coordinates short stand-up meetings to monitor sprint progress and identify potential blockers. During these meetings, each team member briefly explains what they have completed since the previous update, what they plan to work on next, and whether they are experiencing any difficulties. Stand-ups may also be conducted asynchronously through the team's communication channel. The Scrum Master records important updates and follows up on unresolved issues.
-At the end of each sprint, the Scrum Master facilitates a retrospective meeting to evaluate the team's performance and working process. Team members discuss what went well, what problems occurred, and what could be improved in the following sprint. The Scrum Master documents the outcomes and ensures that agreed improvement actions are followed up.
-The Scrum Master is also responsible for monitoring task progress and maintaining the GitHub Projects board according to the workflow defined in Section 5. Reported impediments are tracked and communicated to the relevant team members to prevent unnecessary delays.
-Pull Request reviews are coordinated in accordance with the repository's review rules. The Scrum Master monitors pending reviews and follows up when review requests exceed the 48-hour response target defined in Section 3.